BSgenome - Views

Kasper D. Hansen

Dependencies

This document has the following dependencies:

```
library(BSgenome)
library(BSgenome.Scerevisiae.UCSC.sacCer2)
library(AnnotationHub)
```

Use the following commands to install these packages in R.

```
source("http://www.bioconductor.org/biocLite.R")
biocLite(c("BSgenome",
           "BSgenome.Scerevisiae.UCSC.sacCer2", "AnnotationHub"))
```

Corrections

Improvements and corrections to this document can be submitted on its GitHub in its repository.

Overview

We continue our treatment of *Biostrings* and *BSgenome*

Other Resources

Views

`Views` are used when you have a single big object (think chromosome or other massive dataset) and you need to deal with (many) subsets of this object. `Views` are not restricted to genome sequences; we will discuss `Views` on other types of objects in a different session.

Technically, a `Views` is like an `IRanges` couple with a pointer to the massive object. The `IRanges` contains the indexes. Let's look at `matchPattern` again:

```
library("BSgenome.Scerevisiae.UCSC.sacCer2")
dnaseq <- DNAString("ACGTACGT")
vi <- matchPattern(dnaseq, Scerevisiae$chrI)
vi
```

```
##   Views on a 230208-letter DNAString subject
## subject: CCACACCACACCCACACACCCACACACCACAC...GTGGGTGTGGTGTGGGTGTGGTGTG
## views:
##     start   end width
## [1] 57932 57939     8 [ACGTACGT]
```

We can get the `IRanges` component by

```
ranges(vi)
```

```
## IRanges of length 1
##     start   end width
## [1] 57932 57939     8
```

The `IRanges` gives us indexes into the underlying subject (here chromosome I). To be clear, compare these two:

```
vi
```

```
##   Views on a 230208-letter DNAString subject
## subject: CCACACCACACCCACACACCCACACACCACAC...GTGGGTGTGGTGTGGGTGTGGTGTG
## views:
##     start   end width
## [1] 57932 57939     8 [ACGTACGT]
```

```
Scerevisiae$chrI[ start(vi):end(vi) ]
```

```
##   8-letter "DNAString" instance
## seq: ACGTACGT
```

The `Views` object also look a bit like a `DNAStringSet`; we can do things like

```
alphabetFrequency(vi)
```

```
##      A C G T M R W S Y K V H D B N - + .
## [1,] 2 2 2 2 0 0 0 0 0 0 0 0 0 0 0 0 0 0
```

The advantage of `Views` is that they don't duplicate the sequence information from the subject; all they keep track of are indexes into the subject (stored as `IRanges`). This makes it very (1) fast, (2) low-memory and makes it possible to do things like

```
shift(vi, 10)
```

```
##   Views on a 230208-letter DNAString subject
## subject: CCACACCACACCCACACACCCACACACCACAC...GTGGGTGTGGTGTGGGTGTGGTGTG
## views:
##     start   end width
## [1] 57942 57949     8 [AAGCTTTG]
```

where we now get the sequence 10 bases next to the original match. This could not be done if all we had were the bases of the original subsequence.

`Views` are especially powerful when there are many of them. A usecase I often have are the set of all exons (or promoters) of all genes in the genome. You can use `GRanges` as `Views` as well. Lets look at the hits from `vmatchPattern`.

```
gr <- vmatchPattern(dnaseq, Scerevisiae)
vi2 <- Views(Scerevisiae, gr)
```

Now, let us do something with this. First let us get gene coordinates from *AnnotationHub*.

```
ahub <- AnnotationHub()
qh <- query(ahub, c("sacCer2", "genes"))
qh
```

```
## AnnotationHub with 2 records
## # snapshotDate(): 2015-08-17
## # $dataprovider: UCSC
## # $species: Saccharomyces cerevisiae
## # $rdataclass: GRanges
## # additional mcols(): taxonomyid, genome, description, tags,
## #   sourceurl, sourcetype
## # retrieve records with, e.g., 'object[["AH7048"]]'
##
##            title
##   AH7048 | SGD Genes
##   AH7049 | Ensembl Genes
```

```
genes <- qh[[which(qh$title == "SGD Genes")]]
genes
```

```
## GRanges object with 6717 ranges and 5 metadata columns:
##          seqnames            ranges strand   |         name     score
##             <Rle>         <IRanges>  <Rle>   | <character> <numeric>
##      [1]     chrI [130802, 131986]      +   |     YAL012W         0
##      [2]     chrI [   335,    649]      +   |     YAL069W         0
##      [3]     chrI [   538,    792]      +   |   YAL068W-A         0
##      [4]     chrI [  1807,   2169]      -   |     YAL068C         0
##      [5]     chrI [  2480,   2707]      +   |   YAL067W-A         0
##      ...      ...               ...    ... ...         ...       ...
##   [6713]  chrXIII [923492, 923800]      -   |     YMR326C         0
##   [6714]  2micron [   252,   1523]      +   |      R0010W         0
##   [6715]  2micron [  1887,   3008]      -   |      R0020C         0
##   [6716]  2micron [  3271,   3816]      +   |      R0030W         0
##   [6717]  2micron [  5308,   6198]      -   |      R0040C         0
##              itemRgb            thick        blocks
##          <character>        <IRanges> <IRangesList>
##      [1]        <NA> [130802, 131986]     [1, 1185]
##      [2]        <NA> [   335,    649]      [1, 315]
##      [3]        <NA> [   538,    792]      [1, 255]
##      [4]        <NA> [  1807,   2169]      [1, 363]
##      [5]        <NA> [  2480,   2707]      [1, 228]
##      ...         ...               ...          ...
##   [6713]        <NA> [923492, 923800]      [1, 309]
##   [6714]        <NA> [   252,   1523]     [1, 1272]
##   [6715]        <NA> [  1887,   3008]     [1, 1122]
##   [6716]        <NA> [  3271,   3816]      [1, 546]
##   [6717]        <NA> [  5308,   6198]      [1, 891]
##   -------
##   seqinfo: 18 sequences from sacCer2 genome
```

Let us compute the GC content of all promoters in the yeast genome.

```
prom <- promoters(genes)
head(prom, n = 3)
```

```
## GRanges object with 3 ranges and 5 metadata columns:
##       seqnames            ranges strand |        name     score     it
##          <Rle>         <IRanges>  <Rle> | <character> <numeric> <chara
##   [1]     chrI [128802, 131001]      + |     YAL012W         0
##   [2]     chrI [ -1665,    534]      + |     YAL069W         0
##   [3]     chrI [ -1462,    737]      + |   YAL068W-A         0
##                  thick        blocks
##              <IRanges> <IRangesList>
##   [1] [130802, 131986]     [1, 1185]
##   [2] [   335,    649]      [1, 315]
##   [3] [   538,    792]      [1, 255]
##   -------
##   seqinfo: 18 sequences from sacCer2 genome
```

We get a `warning` that some of these promoters are out-of-band (see the the second and third element in the `prom` object; they have negative values for their ranges). We clean it up and continue

```
prom <- trim(prom)
promViews <- Views(Scerevisiae, prom)
gcProm <- letterFrequency(promViews, "GC", as.prob = TRUE)
head(gcProm)
```

```
##            G|C
## [1,] 0.4668182
## [2,] 0.4868914
## [3,] 0.4572592
## [4,] 0.3731818
## [5,] 0.3859091
## [6,] 0.3309091
```

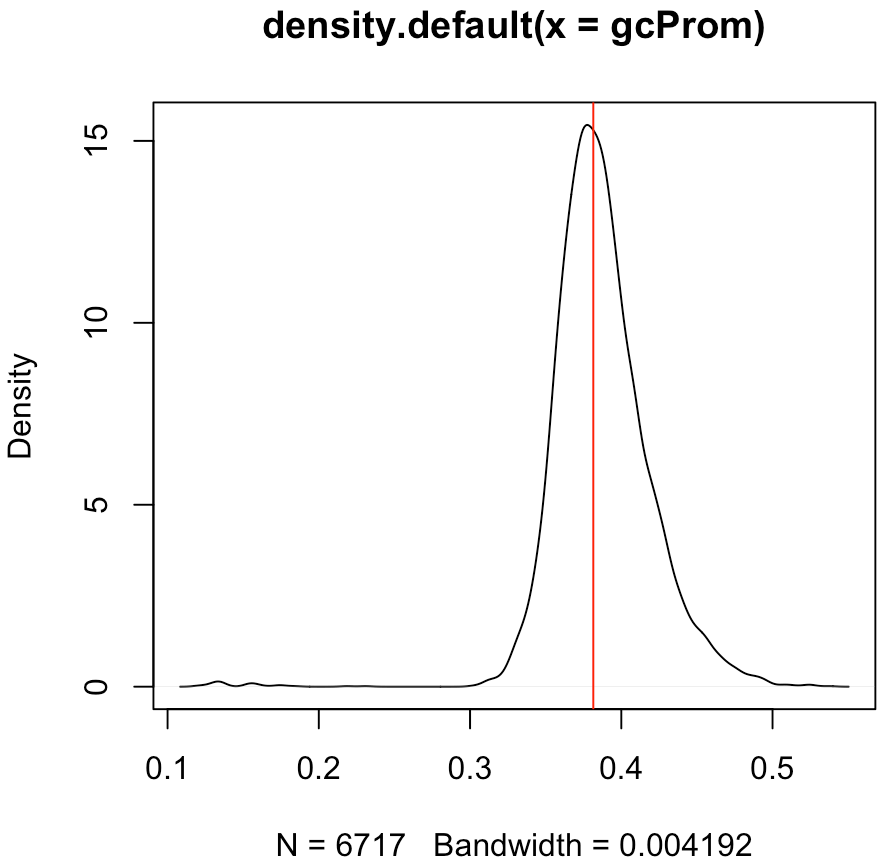
In the previous ***Biostrings*** session we computed the GC content of the yeast genome. Let us do it again, briefly

```
params <- new("BSParams", X = Scerevisiae, FUN = letterFrequency, simpli
gccontent <- bsapply(params, letters = "GC")
gcPercentage <- sum(gccontent) / sum(seqlengths(Scerevisiae))
gcPercentage
```

```
## [1] 0.3814872
```

Let us compare this genome percentage to the distribution of GC content for promoters

```
plot(density(gcProm))
abline(v = gcPercentage, col = "red")
```



At first glance, the GC content of the promoters is not very different from the genome-wide GC content (perhaps shifted a bit to the right).

SessionInfo

```
## R version 3.2.1 (2015-06-18)
## Platform: x86_64-apple-darwin13.4.0 (64-bit)
## Running under: OS X 10.10.5 (Yosemite)
##
## locale:
## [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
##
## attached base packages:
## [1] stats4    parallel  methods   stats     graphics  grDevices utils
## [8] datasets  base
##
## other attached packages:
##  [1] AnnotationHub_2.0.3
##  [2] BSgenome.Scerevisiae.UCSC.sacCer2_1.4.0
##  [3] BSgenome_1.36.3
##  [4] rtracklayer_1.28.8
##  [5] Biostrings_2.36.3
##  [6] XVector_0.8.0
##  [7] GenomicRanges_1.20.5
##  [8] GenomeInfoDb_1.4.2
##  [9] IRanges_2.2.7
## [10] S4Vectors_0.6.3
## [11] BiocGenerics_0.14.0
## [12] BiocStyle_1.6.0
## [13] rmarkdown_0.7
##
## loaded via a namespace (and not attached):
##  [1] Rcpp_0.12.0                  BiocInstaller_1.18.4
##  [3] formatR_1.2                  futile.logger_1.4.1
##  [5] bitops_1.0-6                 futile.options_1.0.0
##  [7] tools_3.2.1                  zlibbioc_1.14.0
##  [9] digest_0.6.8                 evaluate_0.7.2
## [11] RSQLite_1.0.0                shiny_0.12.2
## [13] DBI_0.3.1                    curl_0.9.2
## [15] yaml_2.1.13                  stringr_1.0.0
## [17] httr_1.0.0                   knitr_1.11
## [19] Biobase_2.28.0               R6_2.1.0
## [21] AnnotationDbi_1.30.1         XML_3.98-1.3
## [23] BiocParallel_1.2.20          lambda.r_1.1.7
## [25] magrittr_1.5                 Rsamtools_1.20.4
## [27] htmltools_0.2.6              GenomicAlignments_1.4.1
## [29] xtable_1.7-4                 mime_0.3
## [31] interactiveDisplayBase_1.6.0 httpuv_1.3.3
## [33] stringi_0.5-5                RCurl_1.95-4.7
```